İstanbul Arel University
Computer Engineering Department

READ Refinement and First SaaS Prototype

Date: April 30, 2026, **Due Date:** May 6, 2026, Wednesday

🎯 Goal

In this homework, each project team will continue improving its requirements artifacts and will also build the first working prototype of its SaaS application based on those requirements.

Your team has already produced the first version of its requirements using Visual Paradigm and Word, following the READ template. In this homework, you will both refine those artifacts and begin implementing your software system.

The homework has two parts, each worth 50 points.

PART 1

📌 READ Improvement and Visual Paradigm Refinement (50 points)

In this part, you will continue improving your project requirements and modeling artifacts.

Goal
Refine and strengthen your READ work so that it is more complete, more consistent, and more suitable for implementation.

Submission

The **project leader** must upload **one zip file** to Google Classroom using the following naming format: **NameSurname_HW5.zip**

This zip file must include the following four files:

- **ShortenedProjectName_READ3.docx**
- **ShortenedProjectName_READ3.pdf**
- **ShortenedProjectName_READ3.vpp**
- **ShortenedProjectName_READ_VP3.pdf**

Important requirements

- Your **Visual Paradigm project directory structure** must be organized according to the provided **READ template**.
- The two PDF files should be **highly consistent in structure and content**.
- The Word document and the Visual Paradigm models must describe the **same system**, the **same requirements**, and the **same scope**.
- The revised version should clearly show improvement in requirement quality, completeness, correctness, consistency, prioritization, and modeling quality.

What should be improved

You are expected to improve, where needed:

- requirement clarity and precision
- consistency between textual and modeled requirements
- use cases and supporting models
- functional and non-functional requirements
- priorities and traceability
- missing, weak, ambiguous, or conflicting requirements
- organization of the READ artifact based on the template

PART 2

📌 First SaaS Application Prototype on the Cloud (50 points)

(Updated Version — Modernized Architectural Expectations)

🎯 Goal

In this part, each project team will design, implement, and deploy the **first working prototype** of its system as a **cloud-based SaaS application**.

The primary objective is to demonstrate the ability to move from:

Requirements → User Stories → Architecture → Implementation → Automated Tests

This prototype is expected to be a **small but complete vertical slice** of your system, grounded in your current READ artifacts.

🧱 Minimum Implementation Scope

Your prototype must include:

- ✓ One **home page**
- ✓ One or two **additional functional pages or features**
- ✓ Deployment on a **cloud platform or hosting provider**
- ✓ Implementation of **core features derived from your requirements**
- ✓ A **working end-to-end flow** (UI → backend → data → response)

📐 Architectural Requirements

Your implementation must clearly demonstrate the following architectural principles:

1. Client–Server Architecture

- Separation between **client (UI)** and **server (backend logic)**

2. 3-Tier Architecture (Mandatory)
Your system must include:

Layer	Responsibility
Presentation Layer	User interface (web page, simple frontend, or API interface)
Logic Layer	Business logic, request handling, application behavior
Data Layer	Persistent storage (database or equivalent)

3. MVC or Layered Backend Architecture
You must implement a **clear separation of concerns** in the backend:

- Controller / API Layer → handles requests
- Service / Logic Layer → business rules
- Data / Model Layer → persistence

👉 **Important Clarification**:
MVC is expected **conceptually**, but is **not limited to traditional server-rendered frameworks**.

4. RESTful API Design (Mandatory)
Your system must expose clear and consistent HTTP endpoints.
Examples:

```
GET    /resources
POST   /resources
GET    /resources/{id}
PUT    /resources/{id}
DELETE /resources/{id}
```

Best practices:

- Use appropriate HTTP methods
- Use meaningful resource names
- Return structured JSON responses

5. Cloud Deployment
Your system must be:

- Deployed to a **cloud environment** (e.g., **Heroku, AWS, Azure, GCP, Render, Railway, Vercel, Netlify, PythonAnywhere**, etc.)
- Accessible or demonstrable via:
  - URL
  - or deployment evidence (screenshots, logs)

💡 **Implementation Options (Flexibility)**

You may choose **one of the following approaches**:

Option A — Traditional MVC (Server-Rendered)

- Frameworks: Django, Rails, Spring MVC, etc.
- Server generates HTML views

Option B — Modern API-Based Architecture (Recommended)

- Backend exposes REST APIs (JSON)
- Frontend:
  - simple HTML/JS OR
  - lightweight UI OR
  - API testing interface (e.g., Swagger/Postman)

Option C — Hybrid Approach

- Backend API + minimal UI

👉 **Note**:
Modern API-based implementations are **strongly encouraged** but not mandatory.

🧪 **Development Paradigm (Mandatory)**

Your development must follow:

✓ BDD / TDD Approach
You must demonstrate:

- **User stories**
- **Acceptance criteria (Given / When / Then)**
- **Acceptance tests (e.g., Cucumber or equivalent)**
- **Functional tests**
- **Unit tests**

📦 **Required Outputs for Part 2**

Your submission must include:

1. Architecture Description

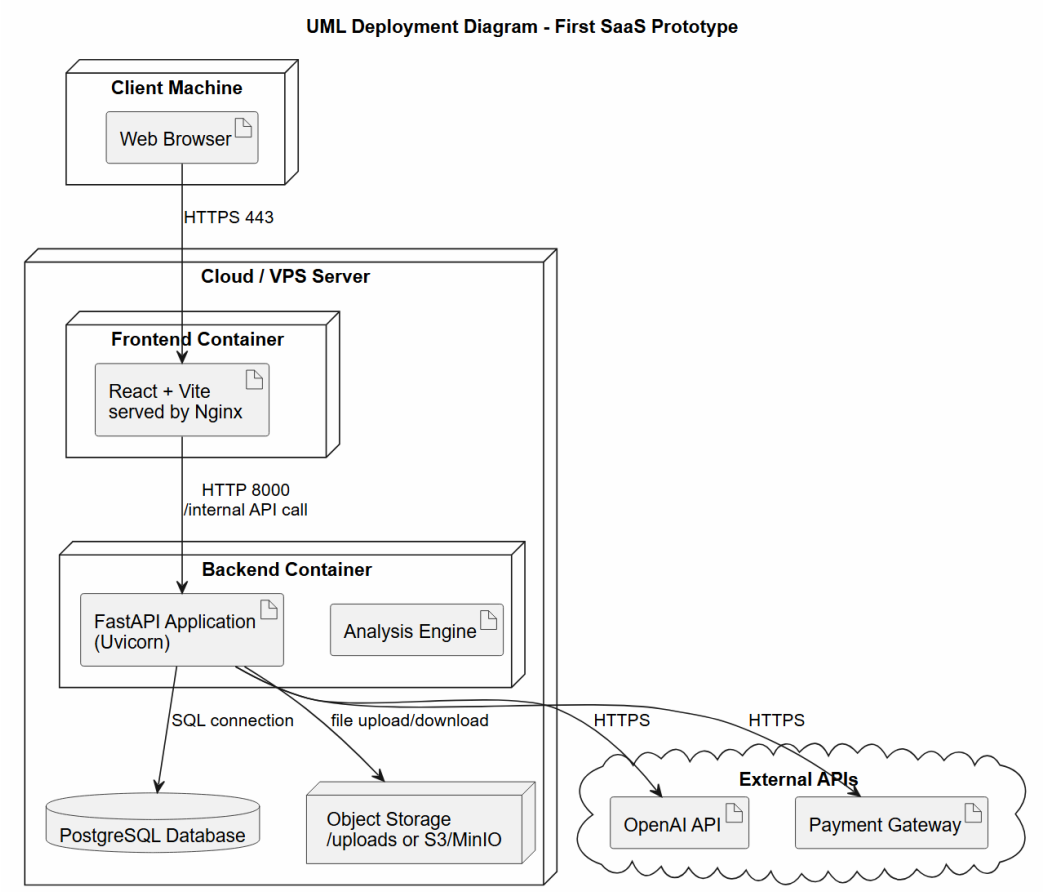
- at least 2 architecture diagrams (diagram + explanation)
- Each diagram must include clear labels, components, and relationships, and must be easily readable when included in your report.
- Must clearly show:
  - layers
  - components
  - interactions

You must include at least 2 architecture diagrams:

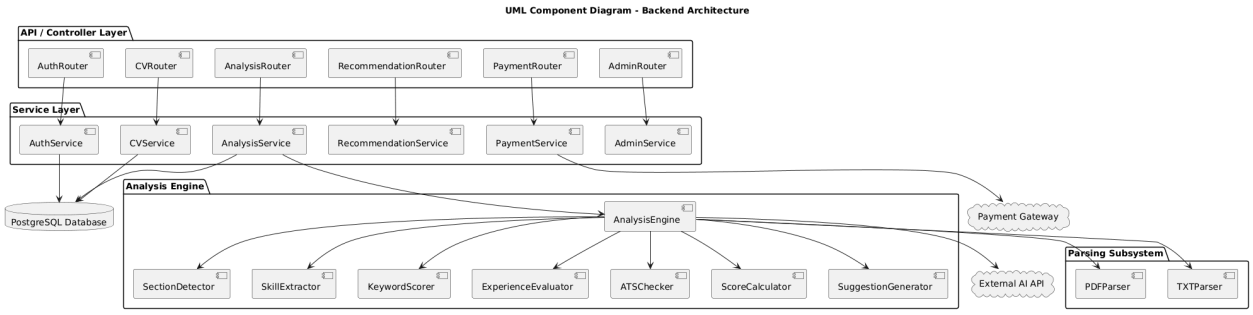
1. UML Deployment Diagram
Shows where the system runs:
client machine, cloud/VPS/server, frontend, backend, database, storage, and external APIs.

2. UML Component or Layered Architecture Diagram
Shows the internal structure of the backend:
API/controller layer, service layer, data layer, analysis/business modules, and external integrations.

A UML deployment diagram example:



A UML component diagram example:



This component diagram may not be readable at this scale. The following page provides clearer and recommended alternatives.

If your component diagram becomes too large, split it into 2–3 smaller readable diagrams, such as: high-level backend layers, analysis engine internals, and API-to-service responsibility map.

Much better **readable** component diagram examples for the above diagram:

Diagram 1 — High-Level Backend Layers

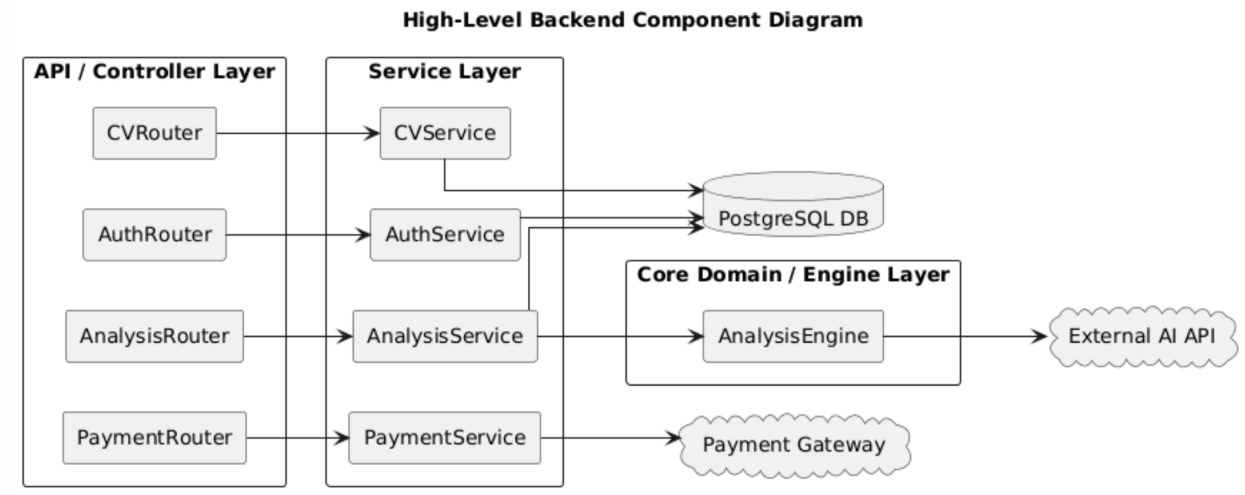


Diagram 2 — Analysis Engine Internal Components

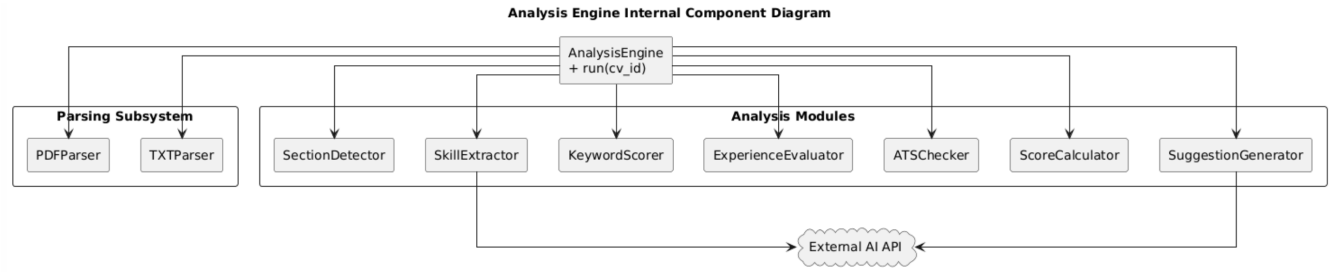
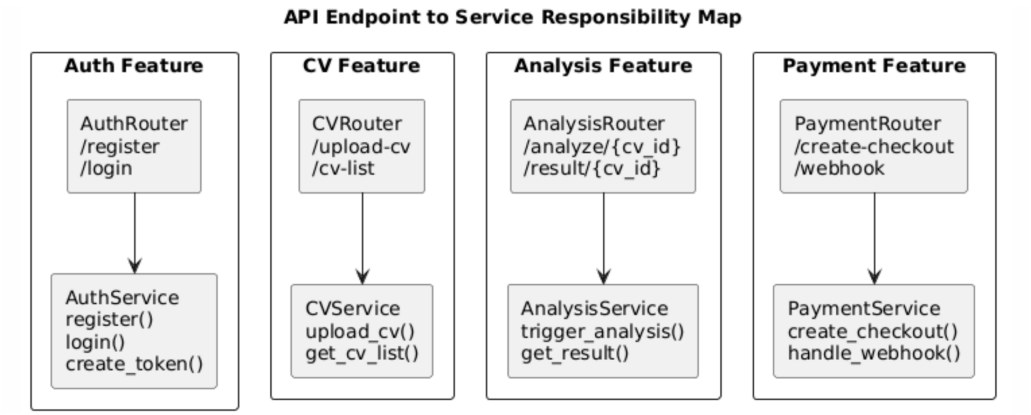


Diagram 3 — API Endpoint to Service Responsibility Map



Important Warning: Make sure that your diagrams must be readable in your reports.

Diagrams may be created using **Visual Paradigm**, **PlantUML**, **draw.io**, or another UML-compatible tool. Diagrams must be exported clearly as PDF, PNG, or SVG.

2. Technology Stack

- Brief justification of technology choices:
  - Programming language(s)
  - Framework(s)
  - Database
  - Cloud platform

3. Implementation Summary (1 page)

- What is implemented
- What works
- What is not yet implemented
- Key design decisions

4. User Stories

- Derived from your requirements
- Small but meaningful set

5. Acceptance Criteria

- Written in BDD format:

```
Given ...
When ...
Then ...
```

6. Automated Tests

- Acceptance tests
- Functional tests
- Unit tests

👉 Include **evidence that tests pass**

7. Working Prototype

- Running system (minimum scope)
- Screenshots or demo evidence

8. Cloud Deployment Evidence

- URL OR
- Screenshots/logs showing deployment

📁 **Submission Format**

Inside your main submission **`NameSurname_HW5.zip`**, include a folder or zip file named: **`FirstSaaSPrototype/`**

This folder should contain:

```
FirstSaaSPrototype/
    ├── source_code/
    ├── test_code/
    ├── architecture_description.pdf
    ├── implementation_summary.pdf
    ├── user_stories.pdf
    ├── acceptance_tests.pdf
    ├── test_results/
    ├── screenshots/
    └── deployment_info.txt
```

⚠️ **Important Notes**

- Your prototype must be based on your **current requirements**
- This is a **vertical slice**, not a full system
- Focus on:
  ✓ correct architecture
  ✓ traceability
  ✓ working system
  ✓ test-supported development

🧠 Final Guidance to Students

A successful submission will clearly demonstrate:

- You understand **software architecture**, not just coding
- You can connect:
  requirements → design → implementation → testing
- You can build a **working SaaS system**, even at small scale

📊 Evaluation Criteria

Part 1 – READ Improvement (50 points)

- Quality of requirements refinement – 10 pts
- Consistency between Word and VP artifacts – 10 pts
- Correct use of the READ template and VP organization – 10 pts
- Quality and completeness of models – 10 pts
- Professionalism, structure, and submission quality – 10 pts

Part 2 – First SaaS Prototype (50 points)

- Correct use of SaaS (3-tier, API, MVC/layered) concepts – 10 pts
- Working cloud-based prototype with required pages – 10 pts
- Quality of initial software architecture and technology choices – 10 pts
- Quality of user stories, acceptance tests, and traceability to requirements – 10 pts
- Quality and evidence of passing functional/unit tests – 10 pts